APPENDIX

A. Parameter Analysis

TABLE I
PARAMETER ANALYSIS FOR CONFLICT TIME LIMIT.

conflict time limit	1	2	3	4	5	10
Total Collisions	344	322	310	306	321	199
Collision Types	6	8	11	11	10	5

TABLE II
PARAMETER ANALYSIS FOR SPATIAL CONFLICT TIME LIMIT.

spatial conflict time limit	5	10	15	20	30
Total Collisions	200	289	310	333	211
Collision Types	6	10	11	10	7

This section presents a parameter analysis for two key parameters: the conflict time limit (t_c) and the spatial time limit (t_s). The results, detailed in Tables I and II, evaluate how variations in these parameters affect the performance, specifically focusing on total collisions and collision types.

The analysis indicates that the model's performance exhibits robustness within certain value ranges for both parameters. For instance, in Table I, performance remains relatively stable for t_c values between 3 and 5. However, performance can change significantly when parameter values are set at the extremes of the tested range (e.g., t_c = 10 in Table I or t_s = 5 in Table II). This behavior confirms that while our method is not overly sensitive to minor adjustments within an optimal window, selecting appropriate values is crucial for achieving desired outcomes.

B. Time Cost Comparison

TABLE III
COMPARISON OF AVERAGE EXECUTION TIME PER SEARCH STEP FOR OUR METHOD, AV-FUZZER [1], MOSAT [2] AND LEGEND [3].

Metrics	Ours	AV-Fuzzer	MOSAT	Legend
time cost (seconds)	59	57	33	46

Table III presents a comparison of the time cost required for a single search step across the evaluated methods. Our approach records an execution time of 59 seconds, which is higher than MOSAT's 33 seconds and Legend's 46 seconds but highly comparable to the 57 seconds observed for AV-Fuzzer. This alignment in computational overhead is expected, as our method is architecturally built upon the AV-Fuzzer framework. Since the optimization of individual search step duration was not the primary objective of our approach, the time cost remains consistent with the underlying baseline while accommodating our proposed two-stage testing method.

C. Experiments with Autoware and CARLA

To verify the effectiveness of our method on different autonomous driving systems and simulators, we conduct additional experiments with Autoware 1.14 [4] using CARLA 0.9.10 [5] simulator, and compare the performance with DriveFuzz [6]. As shown in Table IV, our method significantly outperforms DriveFuzz in both the volume and diversity of discovered failures. Specifically, our approach generates more than twice as many total collisions (194 vs. 89) and identifies a substantially greater variety of collision types (13 vs. 5). In terms of efficiency, our method finds the initial collision faster (5 search steps compared to 11) and requires only half as many steps on average to generate a single collision (3 vs. 6). Furthermore, while we observed that a large portion of the collisions found by DriveFuzz are rear-end crashes initiated by the NPC rather than the ego vehicle, our conflict-driven framework continues to uncover a diverse set of valid ego-vehicle failures. Ultimately, these results validate that our two-stage approach is highly effective and generalizable across different ADS architectures and simulation platforms.

TABLE IV
PERFORMANCE COMPARISON AGAINST DRIVEFUZZ ON TESTING AUTOWARE 1.14 WITH CARLA 0.9.10. EACH EXPERIMENT RUNS FOR 500 SCENARIOS AND REPEATED FOR 3 TIMES.

Metrics	Ours	Drivefuzz
Total Collisions	194	89
Collision Types	13	5
Search Steps for First Collision	5	11
Average Search Steps for One Collision	3	6
Search Steps for All Collision Types	455	84

D. Examples for Collision Types for RQ1

A catalog of all 12 discovered collision types, each with a figure and a detailed description, is provided in the following.

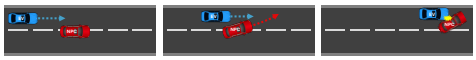


Fig. 1. Example of 1st collision type.

Example 1: shown as Figure 1, the NPC vehicle is initially stopped between two lanes. As the EV approaches, the NPC vehicle begins to slowly merge into the lane occupied by the EV. The EV cannot avoid the NPC vehicle and collides with it. In this case, the Prediction module likely fails to assign sufficient probability to the high-risk action of merging when encountering a vehicle stopped in an ambiguous position between lanes.

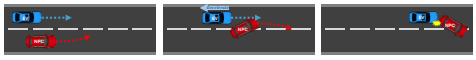


Fig. 2. Example of 2nd collision type.

Example 2: shown as Figure 2, the ego and NPC vehicles are traveling in different lanes. The NPC vehicle merges into the EV's lane and then immediately returns to its original lane. The EV fails to decelerate in time, resulting in a collision with the NPC vehicle. In this case, the Prediction module likely struggles to accurately forecast the trajectory of a highly erratic vehicle performing rapid, successive lane changes, leading to unstable or incorrect planning by the EV.

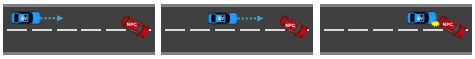


Fig. 3. Example of 3rd collision type.

Example 3: shown as Figure 3, the NPC vehicle is stopped between two lanes, with its front end oriented away from the EV's lane. As the EV passes by, it sideswipes the NPC vehicle from the side, resulting in a collision. In this case, the Planning module likely fails to calculate a safe path with sufficient lateral clearance around the stationary NPC vehicle.

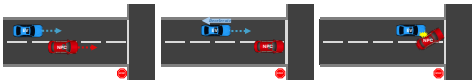


Fig. 4. Example of 4th collision type.

Example 4: shown as Figure 4, the ego and NPC vehicles are traveling in different lanes. As the NPC vehicle approaches an intersection, it suddenly turns its front end toward another lane. Although the EV begins to brake, it does not fully decelerate and ultimately collides with the NPC vehicle. In this case, the Planning module likely fails to execute a sufficiently aggressive emergency braking maneuver in response to the NPC's sudden turn, potentially underestimating the collision risk presented by the rapidly changing situation near the intersection.

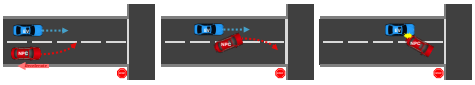


Fig. 5. Example of 5th collision type.

Example 5: shown as Figure 5, the ego and NPC vehicles are traveling in different lanes. As the NPC vehicle approaches an intersection, it decelerates and attempts to merge into the EV's lane. However, midway through the lane change, while straddling the center line, the NPC vehicle abruptly returns to its original lane. The EV cannot avoid the NPC vehicle and collides with it. In this case, the Prediction module fails to cope with the highly erratic and indecisive behavior of the NPC vehicle, as the abrupt reversal of the merge maneuver invalidates the system's short-term trajectory forecast.

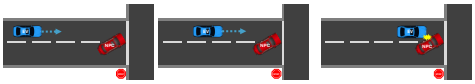


Fig. 6. Example of 6th collision type.

Example 6: shown as Figure 6, the NPC vehicle is stopped at an angle in the intersection. Although the EV decelerates, it does not slow down enough and collides with the NPC vehicle. In this case, the Planning module fails to correctly calculate the required deceleration or stopping distance to avoid the stationary hazard, likely underestimating the risk posed by the NPC vehicle's ambiguous angular position in the intersection.

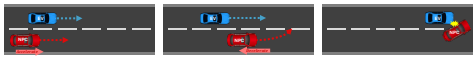


Fig. 7. Example of 7th collision type.

Example 7: shown as Figure 7, the ego and NPC vehicles are initially in different lanes. The NPC vehicle is positioned side-rear relative to the EV. It then accelerates, moves to side-front position, and merges in front of the EV and decelerates. The EV is unable to slow down in time and collides with the NPC vehicle. In this case, the Planning module fails to re-establish a safe following distance immediately after the aggressive cut-in maneuver, making a collision unavoidable when the NPC abruptly decelerates in front of the EV.

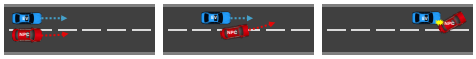


Fig. 8. Example of 8th collision type.

Example 8: shown as Figure 8, the NPC vehicle is traveling in an adjacent lane near the center line and gradually moves closer to the EV's lane. As the NPC vehicle partially enters the lane occupied by the EV, the EV fails to decelerate and collides with the NPC vehicle. In this case, the Planning module fails to apply a sufficient safety margin or react decisively to the NPC's gradual lane drift, potentially because its thresholds for lane intrusion are set too high to prevent overly sensitive reactions in normal traffic.



Fig. 9. Example of 9th collision type.

Example 9: shown as Figure 9, the first NPC vehicle is stationary in the same lane as the EV. A second NPC vehicle initially merges into the EV's lane and, upon approaching the first NPC vehicle, attempts to overtake it by merging back to its original lane and then braking. The EV does not slow down in time, resulting in a collision with the second NPC vehicle. In this case, the Prediction module fails to accurately forecast the complex, multi-stage maneuver of the second NPC, specifically its decision to brake after changing lanes to overtake another vehicle.

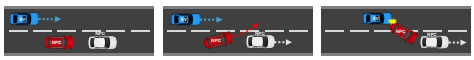


Fig. 10. Example of 10th collision type.

Example 10: shown as Figure 10, both NPC vehicles are initially in different lanes from the EV. The first NPC vehicle is stopped, while the second NPC vehicle is waiting in another lane. The second NPC vehicle then begins to merge into the EV's lane. However, once the first NPC vehicle starts moving, the second NPC vehicle aborts its lane change and merges back. The EV fails to decelerate in time and collides with the second NPC vehicle. In this case, the Prediction module fails to model the complex, coupled interaction between the two NPCs, specifically by not anticipating that the second NPC's

decision to merge and then abort is dependent on the first NPC starting to move.

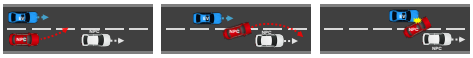


Fig. 11. Example of 11th collision type.

Example 11: shown as Figure 11, the first NPC vehicle is stopped in a different lane from the EV. The second NPC vehicle, also in a different lane from the EV, completes a lane change to overtake the first NPC vehicle but remains at a lower speed than the EV. As a result, the EV does not decelerate in time and collides with the second NPC vehicle. In this case, the Planning module fails to adjust the EV's speed to match the slower velocity of NPC 2 after it completes the lane change, indicating a failure in the system's longitudinal control or target acquisition logic.

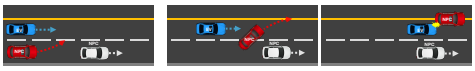


Fig. 12. Example of 12th collision type.

Example 12: shown as Figure 12, the NPC vehicle is traveling with part of its body straddling the yellow center line of the oncoming lane. As the EV passes by, it makes side contact with the NPC vehicle, resulting in a sideswipe collision. In this case, the Planning module fails to calculate a safe trajectory with sufficient lateral clearance while passing the oncoming NPC, underestimating the risk or misjudging the precise boundaries of the vehicle encroaching on its lane.

E. Examples for Extra Collision Types for RQ2

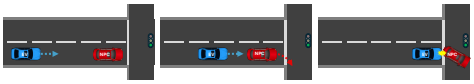


Fig. 13. Example of 13th collision type.

Example 13: shown as Figure 13, the NPC vehicle and the EV are both initially traveling on the same road. When the EV is approaching the intersection, the NPC positions to turn right. As the EV proceeds forward into the intersection and tries to turn right, it doesn't wait for the NPC to complete the turning and collide with the turning NPC. In this case, the Prediction module likely failed to accurately predict the path or turning radius of the NPC vehicle.

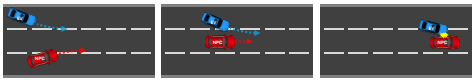


Fig. 14. Example of 14th collision type.

Example 14: shown as Figure 14, the NPC vehicle and the EV are both trying to merge into the same lane from different lanes. The NPC vehicle first completes the lane changing and accelerates, maintaining partial forward overlap with the EV and occupying the target space. The EV fails to yield and continues the lane change maneuver, resulting in a side-impact collision with the NPC vehicle. In this case, the Planning module fails to abort the lane change maneuver when the target space becomes contested, likely because the Prediction module does not accurately model the NPC's defensive acceleration in response to the EV's merge attempt.

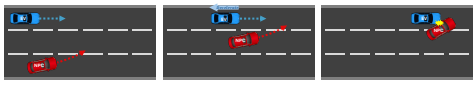


Fig. 15. Example of 15th collision type.

Example 15: shown as Figure 15, the NPC vehicle initially locates two lanes away from the EV. The NPC then initiates a double lane change to move into the EV's lane. As the NPC vehicle executes the second phase of this maneuver, entering the lane directly ahead of or adjacent to the EV, the EV fails to decelerate in time and collides with the NPC vehicle. In this case, the Prediction module fails to recognize the NPC's continuous lateral movement as a double lane change, likely incorrectly anticipating that the NPC would stabilize in the adjacent lane rather than continuing into the EV's lane.

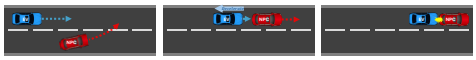


Fig. 16. Example of 16th collision type.

Example 16: shown as Figure 16, the NPC vehicle is initially positioned significantly ahead of the EV in an adjacent lane when it begins changing into the EV's lane. In response to the NPC's maneuver, the EV starts to decelerate. However, after the NPC successfully completes the lane change and occupies the space ahead of the EV, the EV's deceleration proves insufficient to maintain a safe following distance, resulting in a rear-end collision with the NPC. In this case, the Planning module fails to calculate the appropriate deceleration required to match speeds and establish a safe following distance after the NPC enters the EV's lane.

F. Examples of Collision Types for Autoware and CARLA

A catalog of all 13 discovered collision types for Autoware, each with a detailed description of the scenario and underlying system failure, is provided in the following.

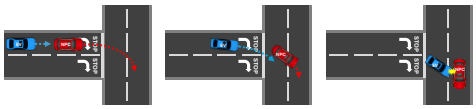


Fig. 17. Example of 1st collision type found in Autoware + Carla experiments.

Example 1: shown as Figure 17, at a dual right-turn T-intersection, the preceding NPC vehicle is executing a right turn. The ego vehicle attempts to turn right while simultaneously performing a lane change to the right, resulting in a side-impact collision with the turning NPC. In this case, the Planning module likely fails to maintain proper lane

discipline during a turning maneuver, executing a hazardous simultaneous lane change and turn that encroaches upon the NPC's occupied space.

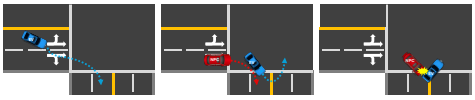


Fig. 18. Example of 2nd collision type found in Autoware + Carla experiments.

Example 2: shown as Figure 18, the ego vehicle successfully completes a right turn at an intersection but then abruptly executes a left U-turn. This erratic maneuver causes a collision with an NPC vehicle that is turning right behind it. In this case, the Planning module experiences a severe routing or trajectory generation failure, initiating an unpredictable and illegal U-turn without adequately assessing rear clearance or trailing traffic.

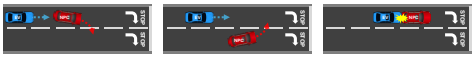


Fig. 19. Example of 3rd collision type found in Autoware + Carla experiments.

Example 3: shown as Figure 19, an NPC vehicle traveling ahead in the same lane performs a lane change and then immediately merges back into its original lane in front of the ego vehicle. The ego vehicle fails to decelerate and crashes directly into the NPC. In this case, the Prediction module likely fails to anticipate the erratic double lane change, or the Planning module is unable to react quickly enough to re-establish a safe following distance against the sudden cut-in.

Example 4: This scenario is structurally identical to the 8th collision type identified during the Apollo evaluation. Please refer to Example 8 in Appendix D for the detailed description and failure analysis.

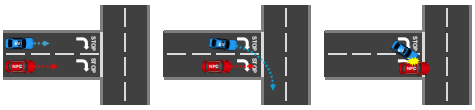


Fig. 20. Example of 5th collision type found in Autoware + Carla experiments.

Example 5: shown as Figure 20, at a dual right-turn T-intersection, the ego vehicle is positioned in the left turning lane. It initiates the right turn prematurely, cutting the corner and colliding with an NPC vehicle in the right turning lane that is preparing to turn. In this case, the Planning module likely fails to compute the correct turning radius for the outer lane, causing the ego vehicle's trajectory to dangerously encroach upon the inner turning lane.

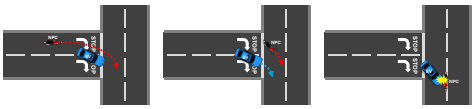


Fig. 21. Example of 6th collision type found in Autoware + Carla experiments.

Example 6: shown as Figure 21, the ego vehicle stops and waits mid-turn at a dual right-turn T-intersection. During this pause, an NPC overtakes the ego vehicle from behind and cuts in front of it. The ego vehicle suddenly accelerates and collides with the NPC. In this case, the Perception or Prediction module likely struggles to detect or accurately forecast the trajectory of the NPC that entered its immediate blind spot during the wait.

Example 7: This scenario is structurally identical to the 3rd collision type identified during the Apollo evaluation. Please refer to Example 3 in Appendix D for the detailed description and failure analysis.

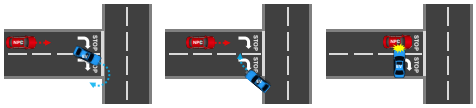


Fig. 22. Example of 8th collision type found in Autoware + Carla experiments.

Example 8: shown as Figure 22, while executing a right turn at a dual right-turn T-intersection, the ego vehicle begins driving in continuous circles. This erratic looping trajectory eventually leads to a collision with an NPC vehicle approaching from behind. In this case, the Planning and Control modules likely suffer a severe trajectory generation loop or localization failure, causing the vehicle to lose its intended path and circle aimlessly within the intersection.

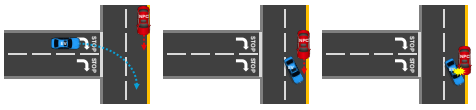


Fig. 23. Example of 9th collision type found in Autoware + Carla experiments.

Example 9: shown as Figure 23, at a dual right-turn T-intersection, the ego vehicle approaches a stop line but fails to wait for crossing traffic. As the ego vehicle proceeds to turn right, it is struck by an NPC vehicle traveling horizontally through the intersection. In this case, the Planning module fails to respect the stop line rules or properly yield to the right-of-way of the crossing vehicles.

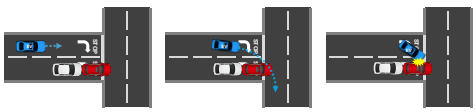


Fig. 24. Example of 10th collision type found in Autoware + Carla experiments.

Example 10: shown as Figure 24, multiple NPC vehicles are queued and stationary in the right-turn lane ahead. The ego vehicle fails to decelerate in time and crashes directly into the rear of the stationary queue. In this case, the Perception module likely suffers a delayed recognition of the static obstacles, or the Planning module miscalculates the required braking distance for the multi-vehicle queue.

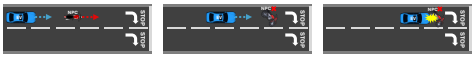


Fig. 25. Example of 11th collision type found in Autoware + Carla experiments.

Example 11: shown as Figure 25, a motorcycle NPC has crashed and is lying stationary on the ground directly in front of the ego vehicle at an intersection. As the ego vehicle proceeds forward, it fails to recognize the fallen motorcycle and its rider, driving directly over them. In this case, the Perception module exhibits a critical failure in identifying non-standard, low-profile stationary obstacles or debris on the roadway.



Fig. 26. Example of 12th collision type found in Autoware + Carla experiments.

Example 12: shown as Figure 26, the ego vehicle is navigating a curved road and collides with preceding NPCs traveling in the same lane. In this case, the Planning module likely fails to adjust its speed or maintain a sufficient longitudinal safety margin given the complex lateral dynamics and potentially reduced forward visibility of the curved trajectory.

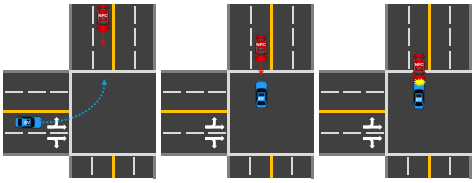


Fig. 27. Example of 13th collision type found in Autoware + Carla experiments.

Example 13: shown as Figure 27, the ego vehicle executes a left turn at an intersection but incorrectly enters the opposing traffic lane, driving against the flow of traffic. This results in a direct head-on collision with an oncoming NPC vehicle. In this case, the Routing or Planning module generates a critically flawed path that violates fundamental lane directionality rules within the intersection.

REFERENCES

[1] G. Li, Y. Li, S. Jha, T. Tsai, M. Sullivan, S. K. S. Hari, Z. Kalbarczyk, and R. Iyer, "Av-fuzzer: Finding safety violations in autonomous driving systems," in *2020 IEEE 31st international symposium on software reliability engineering (ISSRE)*. IEEE, 2020, pp. 25–36.

[2] H. Tian, Y. Jiang, G. Wu, J. Yan, J. Wei, W. Chen, S. Li, and D. Ye, "Mosat: finding safety violations of autonomous driving systems using multi-objective genetic algorithm," in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2022, pp. 94–106.

[3] S. Tang, Z. Zhang, J. Zhou, L. Lei, Y. Zhou, and Y. Xue, "Legend: A top-down approach to scenario generation of autonomous driving systems assisted by large language models," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 1497–1508.

[4] S. Kato, S. Tokunaga, Y. Maruyama, S. Maeda, M. Hirabayashi, Y. Kitsukawa, A. Monrroy, T. Ando, Y. Fujii, and T. Azumi, "Autoware on board: Enabling autonomous vehicles with embedded systems," in *2018 ACM/IEEE 9th International Conference on Cyber-Physical Systems (ICCPS)*. IEEE, 2018, pp. 287–296.

[5] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "Carla: An open urban driving simulator," in *Conference on robot learning*. PMLR, 2017, pp. 1–16.

[6] S. Kim, M. Liu, J. J. Rhee, Y. Jeon, Y. Kwon, and C. H. Kim, "Drivefuzz: Discovering autonomous driving bugs through driving quality-guided fuzzing," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2022, pp. 1753–1767.